

MODULE 1: VARIABLES AND DATA TYPES

1.1 What is Programming?

Programming is the process of writing instructions that a computer can execute to perform specific tasks. A programming language provides the syntax and rules for writing these instructions.

1.2 Variables

A variable is a named storage location in computer memory that holds a value. Think of it as a labeled box.

Example in Python:

```
name = 'Alice'    # string
age = 20          # integer
gpa = 3.85        # float
is_enrolled = True # boolean
```

Naming Rules:

- Must start with a letter or underscore
- Can contain letters, numbers, underscores
- Case-sensitive (age != Age != AGE)
- Cannot use Python keywords (if, for, while, etc.)

1.3 Data Types

int - whole numbers: 42, -7, 0
float - decimal numbers: 3.14, -0.5
str - text: 'hello', "world"
bool - True or False
NoneType - represents absence of value

Type Functions:

```
type(x) - returns the type of x
int(x)   - converts x to integer
float(x) - converts x to float
str(x)   - converts x to string
```

MODULE 2: CONTROL FLOW

2.1 Conditional Statements

if condition:

 # code if true

elif condition2:

 # code if condition2 true

else:

 # code if all false

Example:

grade = 85

if grade >= 90:

 print('A')

elif grade >= 80:

 print('B')

elif grade >= 70:

 print('C')

else:

 print('F')

Comparison Operators:

== equal to

!= not equal to

> greater than

< less than

>= greater or equal

<= less or equal

Logical Operators:

and - both conditions must be true

or - at least one condition must be true

not - inverts the condition

2.2 Loops

For Loop (iterates over a sequence):

for i in range(5): # 0, 1, 2, 3, 4

 print(i)

for char in 'hello': # iterates over characters

 print(char)

fruits = ['apple', 'banana', 'cherry']

for fruit in fruits:

 print(fruit)

While Loop (repeats while condition is true):

```
count = 0
while count < 5:
    print(count)
    count += 1
```

Loop Control:

break - exit the loop immediately
continue - skip to next iteration

Example:

```
for i in range(10):
    if i == 3:
        continue # skip 3
    if i == 7:
        break # stop at 7
    print(i) # prints: 0,1,2,4,5,6
```

MODULE 3: FUNCTIONS

3.1 Defining Functions

A function is a reusable block of code that performs a specific task. Functions help organize code and avoid repetition.

```
def function_name(parameters):
    # code block
    return result
```

Example:

```
def calculate_average(numbers):
    total = sum(numbers)
    count = len(numbers)
    return total / count
```

```
result = calculate_average([85, 90, 78, 92])
print(result) # 86.25
```

3.2 Parameters and Arguments

```
def greet(name, greeting='Hello'): # default parameter
    return f'{greeting}, {name}!'
```

```

greet('Alice')          # 'Hello, Alice!'
greet('Bob', 'Good morning') # 'Good morning, Bob!'

def add_all(*args):      # variable arguments
    return sum(args)

add_all(1, 2, 3, 4) # 10

```

3.3 Scope

```

def my_function():
    local_var = 'I am local' # local scope
    print(global_var)        # can read global

global_var = 'I am global'   # global scope
my_function()
print(global_var)            # works
# print(local_var)          # NameError!

```

MODULE 4: DATA STRUCTURES

4.1 Lists

```

my_list = [1, 2, 3, 4, 5]
my_list.append(6)      # add to end
my_list.insert(0, 0)   # insert at index
my_list.remove(3)      # remove first occurrence
popped = my_list.pop() # remove and return last
my_list[0]             # access by index
my_list[1:3]           # slice: [2, 3]
len(my_list)           # length
3 in my_list           # membership test: True

```

4.2 Dictionaries

```

student = {
    'name': 'Alice',
    'age': 20,
    'gpa': 3.85
}
student['name']          # 'Alice'
student['major'] = 'CS'  # add new key
del student['age']       # delete key

```

```
student.keys()          # dict_keys(['name', 'gpa'])
student.values()        # dict_values(['Alice', 3.85])
student.get('name', 'N/A') # safe access
```

4.3 List Comprehensions

```
squares = [x**2 for x in range(10)]
evens = [x for x in range(20) if x % 2 == 0]
words = [w.upper() for w in ['hello', 'world']]
```

MODULE 5: FILE I/O AND ERROR HANDLING

5.1 Reading and Writing Files

```
# Writing
with open('output.txt', 'w') as f:
    f.write('Hello, World!\n')
    f.write('Line two\n')

# Reading
with open('output.txt', 'r') as f:
    content = f.read()    # read all
    lines = f.readlines() # list of lines
    for line in f:        # iterate line by line
        print(line.strip())
```

5.2 Error Handling

```
try:
    result = 10 / 0
except ZeroDivisionError:
    print('Cannot divide by zero!')
except Exception as e:
    print(f'Unexpected error: {e}')
else:
    print('No errors occurred')
finally:
    print('This always runs')
```